Practice Set: C++ Data Types & Type Qualifiers

Section 1: Practice Questions (1–10)

Question 1: Define what a **Data Type** is in C++. Explain the two critical pieces of information a data type provides to the compiler upon variable declaration.

Question 2: Draw or outline the complete **Classification Hierarchy** of C++ Data Types, showing how primary, derived, and user-defined data types are categorized.

Question 3: Fill in the missing values in the table below based on standard modern system memory allocations:

Primary Data Type	**Typical Memory Size (Bytes)**	**Used For**
bool	*(a)*	Boolean truth values (true/false)
char	1 Byte	*(b)*
int	*(c)*	Whole numbers (no decimals)
float	4 Bytes	*(d)*
double	*(e)*	Double-precision floating-point numbers
void	0 Bytes	*(f)*

Question 4: Explain why comparing floating-point numbers (float or double) directly using the == relational operator is risky in C++.

Question 5: Differentiate between **Derived Data Types** and **User-Defined Data Types**. Provide two examples of each type.

Question 6: What is a **Type Qualifier** in C++? Describe the specific memory or behavior impact of the qualifiers short, long, signed, unsigned, const, and volatile.

Question 7: Explain **Unsigned Integer Wrap-around** (Overflow behavior) using an 8-bit unsigned integer example (0 to 255). What value does an 8-bit unsigned variable hold after incrementing past 255?

Question 8: Explain the **Unsigned Underflow Bug**. What happens to an unsigned int variable initialized to 0 when it is decremented (x--)?

Question 9: How does the memory size of a **Pointer** variable (type*) behave on a 32-bit Operating System architecture versus a 64-bit Operating System architecture?

Question 10: Write a complete C++ program that declares and initializes a short int, a long long int, an unsigned int, and a const double, and prints each variable's value along with its memory size in bytes using the sizeof operator.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): What is the exact difference in decimal digit precision between a float and a double in C++?

Question 12 (Viva): How does a Reference (type&) differ conceptually from a Pointer (type*) in C++?

Question 13 (Viva): How does a standard C++ enum differ from a modern enum class?

Question 14 (Viva): What is the main structural memory difference between a struct and a union in C++?

Question 15 (Viva): What does the volatile type qualifier tell the compiler during code optimization?

Question 16 (Viva): Does the void data type allocate any physical memory in RAM? Where is void typically utilized in program code?

Question 17 (Viva): Why does an unsigned int have double the maximum positive range of a signed int while occupying the same 4 bytes of RAM memory space?

Question 18 (Viva): What primary data type was natively introduced in C++ that was absent in standard C?

Question 19 (Viva): What is the valid numerical range of an 8-bit unsigned char versus a standard signed char?

Question 20 (Viva): What is the purpose of the using alias syntax in modern C++ compared to typedef?

Answers & Explanations

Answer: 01

A **Data Type** is an explicit instruction to the compiler that specifies how to handle a declared variable.

The two critical pieces of information it provides are:

Data Nature: What kind of value is stored (e.g., text, whole numbers, decimals, truth values).

Memory Allocation: Exactly how much memory space (in bytes) to reserve in RAM and how the underlying bit pattern should be interpreted.

Answer: 02

C++ Data Types Hierarchy:

Primary / Built-in: void, bool, char (and wchar_t), int, float, double.

Derived Types: Array (std::array), Pointer (int*), Reference (int&), Function.

User-Defined Types: struct, union, enum / enum class, typedef / using alias.

Answer: 03

(a) 1 Byte

(b) Single characters or ASCII codes

(c) 4 Bytes (32 bits)

(d) Single-precision floating-point numbers

(e) 8 Bytes (64 bits)

(f) Absence of type / empty return

Answer: 04 Floating-point numbers (float and double) cannot represent all decimal fractions with exact precision due to IEEE 754 binary floating-point representation limits. Because rounding errors occur during calculation, comparing floating-point values using == can fail unexpectedly.

Answer: 05

Derived Data Types: Data types constructed by referencing, pointing to, or grouping existing basic primary data types. *Examples:* Pointers (int*), References (int&), Arrays.

User-Defined Data Types: Custom data structures created by the developer to group complex real-world data under a single logical unit. *Examples:* struct, union, enum class.

Answer: 06

A **Type Qualifier** modifies the size, range, sign interpretation, or mutability of a base data type.

short: Reduces memory footprint (typically 2 Bytes for int).

long: Expands memory capacity and range (4 or 8 Bytes).

signed: Allows both positive and negative values (default for int and char).

unsigned: Restricts the variable to non-negative values (0 and positive), effectively doubling its positive upper limit.

const: Locks the variable's value, making it read-only and preventing modifications after initialization.

volatile: Prevents compiler optimizations on variables that may be altered by external processes or hardware.

Answer: 07

Unsigned integers operate using modular arithmetic (wrap-around logic). An 8-bit unsigned integer can store values from 0 to 255.

If an 8-bit unsigned variable holding 255 is incremented by 1 (+1), it overflows and **wraps around to 0**.

Answer: 08

The **Unsigned Underflow Bug** occurs when an unsigned variable is decremented below its minimum valid threshold of 0.

Because unsigned variables cannot represent negative numbers, decrementing 0 causes the value to wrap around to its maximum possible value. For a 32-bit unsigned int, x-- when x = 0 results in 4,294,967,295.

Answer: 09

Pointer memory size scales automatically based on the CPU/OS architecture.

On a **32-bit OS**, a pointer occupies **4 Bytes**.

On a **64-bit OS**, a pointer occupies **8 Bytes**.

Answer: 10

```
#include <iostream>
int main()
{
   short int age = 20;
   long long int population = 14000000000LL;
   unsigned int totalStudents = 500;
   const double pi = 3.14159;

   std::cout << "Short Int Value: " << age << " | Size: " << sizeof(age) << " bytes" <<
std::endl;
   std::cout << "Long Long Value: " << population << " | Size: " << sizeof(population)
<< " bytes" << std::endl;
   std::cout << "Unsigned Int Value: " << totalStudents << " | Size: " <<
sizeof(totalStudents) << " bytes" << std::endl;
   std::cout << "Const Double Value: " << pi << " | Size: " << sizeof(pi) << " bytes" <<
std::endl;

   return 0;
}
```

Answer: 11 A float provides approximately **7 decimal digits** of precision (occupying 4 bytes), while a double provides approximately **15–17 decimal digits** of precision (occupying 8 bytes)

Answer: 12 A **Pointer** (type*) is an independent variable that stores the raw physical RAM address of another variable. A **Reference** (type&) is an alias or secondary reference name for an already existing variable's memory location.

Answer: 13 A modern enum class is strongly typed and scoped, which prevents unintended implicit integer conversions and global scope pollution. In contrast, a traditional enum exports its named constants directly into the surrounding scope as un-scoped integers.

Answer: 14 In a struct, every member variable is allocated its own separate memory block in RAM. In a union, all member variables share the exact same single memory location, with the total size determined by the largest member.

Answer: 15 The volatile qualifier informs the compiler that a variable's value may be altered unexpectedly by external hardware processes or concurrent threads. This instructs the compiler not to optimize away or cache reads for that variable

Answer: 16 No, void allocates **0 bytes** of memory. It is primarily used to signal an empty return type for functions or to declare untyped generic pointers (void*).

Answer 17 A signed int uses 1 bit to store the sign (+ or -), leaving 31 bits for value representation (-2,147,483,648 to 2,147,483,647). An unsigned int uses all 32 bits strictly for magnitude (0 to 4,294,967,295), doubling the maximum positive upper limit.

Answer: 18 Native boolean support via the bool data type (along with native wide character types like wchar_t).

Answer: 19 An 8-bit unsigned char ranges from **0 to 255**. A standard signed char ranges from **-128 to 127**.

Answer: 20 Both construct custom type aliases. However, the modern using alias syntax (using NewType = OldType;) provides cleaner readability and natively supports templated aliases, making it the preferred approach in modern C++.